

BAB III

MASTER DATA

3.1 CRUD Kategori sebagai Pola Dasar

3.1.1) Tambahkan relasi pada model Kategori

Buka `app/Models/Kategori.php` dan tambahkan satu method relasi:

```
//...

class Kategori extends Model
{
    //...

    public function daftarAlat()
    {
        return $this->hasMany(Alat::class, 'kategori_id');
    }
}
```

Relasi ini akan dipakai untuk menghitung berapa alat yang bernaung di tiap kategori.

3.1.2) Buat Form Request

Pada terminal ketikkan perintah membuat Request :

```
php artisan make:request KategoriRequest
```

Isi file `app/Http/Requests/KategoriRequest.php`:

```
5 use Illuminate\Foundation\Http\FormRequest;
6 use Illuminate\Validation\Rule;
7
8 class KategoriRequest extends FormRequest
9 {
10     public function authorize(): bool
11     {
12         return $this->user()->can('kategori.kelola');
13     }
14
15     public function rules(): array
16     {
17         $kategoriYangDiubah = $this->route('kategori');
18
19         return [
20             'nama' => [
21                 'required',
22                 'string',
23                 'max:100',
24                 Rule::unique('kategori', 'nama')
25                     ->ignore($kategoriYangDiubah),
26             ],
27             'deskripsi' => ['nullable', 'string', 'max:1000'],
28         ];
29     }
30
31     public function messages(): array
32     {
33         return [
34             'nama.required' => 'Nama kategori wajib diisi.',
35             'nama.unique' => 'Nama kategori tersebut sudah terdaftar.',
36             'nama.max' => 'Nama kategori maksimal 100 karakter.',
37         ];
38     }
39 }
```

Tiga hal yang membuat berkas ini penting:

Bagian	Fungsi
authorize()	Lapis kedua pemeriksaan izin, setelah middleware pada route. Bila izin dicabut, permintaan ditolak walau route-nya terlanjur terbuka
rules()	Pemenuhan NFR-05: validasi dilakukan di sisi server, bukan hanya di peramban
->ignore()	Saat mengubah data, nama kategori itu sendiri tidak boleh dianggap duplikat dirinya sendiri

Variabel `$kategoriYangDiubah` bernilai kosong saat menyimpan data baru, dan berisi objek kategori saat mengubah. Satu berkas Form Request karena itu cukup untuk melayani dua keperluan.

3.1.3) Buat controller

Pada terminal buat controller :

```
php artisan make:controller KategoriController --model=Kategori
```

Buka file `app\Http\Controllers\KategoriController.php` :

```
5 use App\Http\Requests\KategoriRequest;
6 use App\Models\Kategori;
7 use Illuminate\Database\QueryException;
8 use Illuminate\Http\Request;
9
10 class KategoriController extends Controller
11 {
12     public function index(Request $request)
13     {
14         $kataKunci = $request->query('cari');
15
16         $daftarKategori = Kategori::withCount('daftarAlat')
17             ->when($kataKunci, function ($query, $kataKunci) {
18                 $query->where('nama', 'like', '% ' . $kataKunci . '%');
19             })
20             ->orderBy('nama')
21             ->paginate(10)
22             ->withQueryString();
23
24         return view('kategori.index', compact('daftarKategori', 'kataKunci'));
25     }
26
27     public function create()
28     {
29         $kategori = new Kategori();
30
31         return view('kategori.form', compact('kategori'));
32     }
33
34     public function store(KategoriRequest $request)
35     {
36         Kategori::create($request->validated());
37
38         return redirect()
39             ->route('kategori.index')
40             ->with('sukses', 'Kategori berhasil ditambahkan.');
```

```

43 public function edit(Kategori $kategori)
44 {
45     return view('kategori.form', compact('kategori'));
46 }
47
48 public function update(KategoriRequest $request, Kategori $kategori)
49 {
50     $kategori->update($request->validated());
51
52     return redirect()
53         ->route('kategori.index')
54         ->with('sukses', 'Kategori berhasil diperbarui.');
```

Tentang blok `try-catch`. Foreign key `alat.kategori_id` memakai `restrictOnDelete()`, jadi MariaDB menolak penghapusan kategori yang masih dipakai. Tanpa penangkapan `QueryException`, pengguna melihat halaman error berwarna merah yang tidak dipahami. Dengan penangkapan ini, ia melihat kalimat yang menjelaskan masalahnya. Inilah cara utuh — aturannya ditegakkan basis data, pesannya disampaikan aplikasi.

Tentang `withQueryString()`. Tanpa method ini, kata kunci pencarian hilang saat pengguna berpindah ke halaman kedua.

3.1.4) Daftarkan route

Tambahkan ke dalam grup `auth` di `routes/web.php`:

```

//...
use App\Http\Controllers\KategoriController;

Route::middleware(['auth'])->group(function () {
    //...
    Route::resource('kategori', KategoriController::class)
        ->except(['show'])
        ->middleware('permission:kategori.kelola');
});
```

Seluruh grup dilindungi `permission:kategori.kelola`. Karena izin itu hanya dimiliki peran admin pada seeder, petugas dan peminjam otomatis menerima HTTP 403 bila mencoba membuka alamat ini.

Nama parameter `{kategori}` harus sama persis dengan nama variabel `Kategori $kategori` di controller. Kesamaan nama inilah yang membuat Laravel otomatis mencarikan datanya dari basis data.

Periksa hasilnya di terminal:

```
php artisan route:list --name=kategori
```

3.1.5) Buat halaman daftar

Buat file `resources/views/kategori/index.blade.php`:

```
1  @extends('layouts.utama')
2
3  @section('judul', 'Daftar Kategori')
4
5  @section('konten')
6      <div class="d-flex justify-content-between align-items-center mb-3">
7          <h4 class="mb-0">Daftar Kategori</h4>
8          <x-tombol-tambah :href="route('kategori.create')" label="Tambah Kategori" />
9      </div>
10
11     <div class="card">
12         <div class="card-body">
13
14             <x-form-pencarian :action="route('kategori.index')" :kataKunci="$kataKunci" />
15
16             <div class="table-responsive">
17                 <table class="table table-striped align-middle">
18                     <thead>
19                         <tr>
20                             <th style="width: 60px">No</th>
21                             <th>Nama</th>
22                             <th>Deskripsi</th>
23                             <th style="width: 110px">Jumlah Alat</th>
24                             <th style="width: 160px">Aksi</th>
25                         </tr>
26                     </thead>
27
28                     <tbody>
29                         @forelse ($daftarKategori as $nomor => $kategori)
30                             <tr>
31                                 <td>{{ $daftarKategori->firstItem() + $nomor }}</td>
32                                 <td>{{ $kategori->nama }}</td>
33                                 <td>{{ $kategori->deskripsi ?: '-' }}</td>
34                                 <td>{{ $kategori->daftar_alat_count }}</td>
35                                 <td>
36                                     <x-tombol-aksi
37                                         :ubah="route('kategori.edit', $kategori)"
38                                         :hapus="route('kategori.destroy', $kategori)"
39                                         pesanHapus="Yakin ingin menghapus kategori {{ $kategori->nama }}?"
40                                     />
41                                 </td>
42                             </tr>
43                         @empty
44                             <tr>
45                                 <td colspan="5" class="text-center text-muted">
46                                     Belum ada data kategori.
47                                 </td>
48                             </tr>
49                         @endforelse
50                     </tbody>
51                 </table>
52             </div>
53
54             {{ $daftarKategori->links() }}
55
56         </div>
57     </div>
58 @endsection
```

Buat file component `resources\views\components\tombol-tambah.blade.php` :

```
1 @props(['href', 'label' => 'Tambah'])
2 <a href="{{ $href }}" class="btn btn-primary">{{ $label }}</a>
3
```

Buat file component `resources\views\components\form-pencarian.blade.php` :

```
1 @props(['action', 'kataKunci' => '', 'nama' => 'cari', 'placeholder' => 'Cari...'])
2
3 <form method="GET" action="{{ $action }}" class="row g-2 mb-3">
4     <div class="col-md-4">
5         <input type="text" name="{{ $nama }}" class="form-control" placeholder="{{ $placeholder }}"
6             value="{{ $kataKunci }}">
7     </div>
8     <div class="col-auto">
9         <button type="submit" class="btn btn-outline-secondary">Cari</button>
10        <a href="{{ $action }}" class="btn btn-outline-secondary">Reset</a>
11    </div>
12 </form>
13
```

Buat file component `resources\views\components\tombol-aksi.blade.php` :

```
1 @props([
2     'lihat' => null,
3     'ubah' => null,
4     'hapus' => null,
5     'pesanHapus' => 'Yakin ingin menghapus data ini?',
6 ])
7
8 <div class="d-flex gap-1">
9     @if ($lihat)
10        <a href="{{ $lihat }}" class="btn btn-sm btn-info">Lihat</a>
11    @endif
12
13    @if ($ubah)
14        <a href="{{ $ubah }}" class="btn btn-sm btn-warning">Ubah</a>
15    @endif
16
17    @if ($hapus)
18        <form action="{{ $hapus }}" method="POST" class="d-inline" onsubmit="return confirm('{{ $pesanHapus }}')">
19            @csrf
20            @method('DELETE')
21            <button type="submit" class="btn btn-sm btn-danger">Hapus</button>
22        </form>
23    @endif
24 </div>
```

Beberapa hal yang perlu dipahami:

- `$daftarKategori->firstItem() + $nomor` membuat penomoran berlanjut di halaman kedua. Kalau hanya memakai `$nomor + 1`, halaman kedua akan mulai dari angka 1 lagi.
- `withCount('daftarAlat')` di controller menghasilkan properti bernama `daftar_alat_count`. Namanya diubah otomatis oleh Laravel dari huruf kapital menjadi garis bawah.
- Tombol hapus dibungkus form dengan `@method('DELETE')`, karena tautan biasa hanya bisa mengirim permintaan GET.
- `{{ $daftarKategori->links() }}` menghasilkan navigasi halaman.

Agar tampilan navigasi halaman memakai gaya Bootstrap, tambahkan baris berikut pada method `boot()` di `app/Providers/AppServiceProvider.php`:

```
//...
use Illuminate\Pagination\Paginator;

class AppServiceProvider extends ServiceProvider
{
    //...
    public function boot(): void
    {
        Paginator::useBootstrapFive();
    }
}
```

3.1.6) Buat halaman form

Buat file `resources/views/kategori/form.blade.php`. Satu berkas dipakai untuk menambah maupun mengubah data.

```
1  @extends('layouts.utama')
2
3  @section('judul', $kategori->exists ? 'Ubah Kategori' : 'Tambah Kategori')
4
5  @section('konten')
6      <div class="row justify-content-center">
7          <div class="col-md-7">
8              <div class="card">
9                  <div class="card-body">
10                     <h5 class="card-title mb-4">
11                         {{ $kategori->exists ? 'Ubah Kategori' : 'Tambah Kategori' }}
12                     </h5>
13
14                     <form method="POST"
15                         action="{{ $kategori->exists ? route('kategori.update', $kategori) : route('kategori.store') }}">
16                         @csrf
17                         @if ($kategori->exists)
18                             @method('PUT')
19                         @endif
20
21                         <x-input name="nama" label="Nama Kategori" :value="$kategori->nama" />
22
23                         <x-textarea name="deskripsi" label="Deskripsi Kategori" :value="$kategori->deskripsi" />
24
25                         <button type="submit" class="btn btn-primary">Simpan</button>
26                         <a href="{{ route('kategori.index') }}" class="btn btn-secondary">Batal</a>
27                     </form>
28                 </div>
29             </div>
30         </div>
31     </div>
32 @endsection
33
```

Buat file component `resources\views\components\textarea.blade.php` :

```
1  @props(['name', 'label', 'type' => 'text', 'value' => ""])
2
3  <div class="mb-3">
4      <label for="{{ $name }}" class="form-label">{{ $label }}</label>
5      <textarea class="form-control @error($name) is-invalid @enderror"
6          id="{{ $name }}"
7          name="{{ $name }}"
8          rows="3"
9          {{ $attributes }}
10         >{{ old($name, $value) }}</textarea>
11      @error($name)
12          <div class="invalid-feedback">
13              {{ $message }}
14          </div>
15      @enderror
16  </div>
```

Kunci dari form gabungan ini adalah `$kategori->exists`. Objek kategori kosong yang dibuat di method `create()` bernilai false, sedangkan objek hasil pencarian bernilai true. Dari satu properti itu ditentukan judul halaman, alamat tujuan form, dan perlu tidaknya `@method('PUT')`.

3.1.7) Aktifkan menu dan uji

Buka `resources/views/layouts/navbar.blade.php`, ganti tautan menu Kategori menambahkan `route('kategori.index')` pada atribut `href`:

```
<!-- ... -->
<div class="collapse navbar-collapse" id="menuUtama">
  <ul class="navbar-nav me-auto">
    @can('kategori.kelola')
      <li class="nav-item">
        <a class="nav-link" href="{{ route('kategori.index') }}">Kategori</a>
      </li>
    @endcan
  </ul>
<!-- ... -->
</div>
```

No	Percobaan	Hasil yang harus muncul
1	Buka menu Kategori	Tampil 4 kategori hasil seeder dengan jumlah alat masing-masing 3
2	Tambah kategori baru bernama "Alat Kebersihan"	Tersimpan, muncul pesan hijau
3	Tambah lagi dengan nama yang sama	Ditolak, muncul pesan "Nama kategori tersebut sudah terdaftar"
4	Ubah nama kategori tanpa mengubah apa pun lalu simpan	Berhasil, tidak dianggap duplikat
5	Hapus kategori "Alat Kebersihan"	Berhasil, muncul pesan hijau
6	Hapus kategori "Alat Ukur"	Ditolak, muncul pesan merah bahwa kategori masih dipakai
7	Masuk sebagai petugas, buka alamat /kategori	HTTP 403

Halaman Daftar Kategori :

Daftar Kategori

Tambah Kategori

Cari... Cari Reset

No	Nama	Deskripsi	Jumlah Alat	Aksi
1	Alat Ukur	Multimeter, jangka sorong, mistar baja	3	<button>Ubah</button> <button>Hapus</button>
2	Perangkat Audio Visual	Proyektor, kamera, tripod	3	<button>Ubah</button> <button>Hapus</button>
3	Perangkat Jaringan	Switch, router, tang crimping	3	<button>Ubah</button> <button>Hapus</button>
4	Perkakas Tangan	Obeng, tang, kunci, palu	3	<button>Ubah</button> <button>Hapus</button>

Request Timeline Views 10 Queries 8 Models 6 Gate 6 Cache 1 12.x 28MB 916ms GET /kategori

Halaman Form Kategori :

Tambah Kategori

Nama Kategori

Deskripsi Kategori

Simpan Batal

Request Timeline Views 5 Queries 6 Models 2 Gate 6 Cache 1 12.x 28MB 490ms GET /kategori/create

3.2 CRUD Alat dengan Relasi dan Unggah Foto

3.2.1) Siapkan penyimpanan File

Edit file `config/filesystems.php` :

```
'disks' => [
    //...
    'gambar' => [
        'driver' => 'local',
        'root' => public_path('gambar'),
        'url' => env('APP_URL') . '/gambar',
        'visibility' => 'public',
        'throw' => false,
    ],
],
```

Folder `public/gambar` harus ada dan bisa ditulis (writable) — kalau belum ada, Laravel biasanya otomatis membuatnya saat `storeAs()` dipanggil, tapi lebih aman disiapkan manual dulu.

`visibility => 'public'` di sini cuma metadata, bukan symlink — karena root disk sudah langsung di `public/`, file otomatis bisa diakses lewat URL tanpa perantara.

Tambahkan `public/gambar/*` ke `.gitignore` kalau tidak mau file upload ikut ter-commit (opsional, tergantung kebutuhan).

3.2.2) Tambahkan relasi pada model Alat

Buka `app/Models/Alat.php`:

```
//...
class Alat extends Model
{
    //...
    public function kategori()
    {
        return $this->belongsTo(Kategori::class, 'kategori_id');
    }

    public function detailPeminjaman()
    {
        return $this->hasMany(DetailPeminjaman::class, 'alat_id');
    }
}
```

Relasi `kategori()` memakai `belongsTo` karena tabel `alat` yang menyimpan kolom `kategori_id`. Aturan sederhananya: tabel yang memegang foreign key selalu memakai `belongsTo`.

3.2.3) Buat Form Request

Pada terminal buat Request :

```
php artisan make:request AlatRequest
```

Buka file `app\Http\Requests\AlatRequest.php` :

```
5 use Illuminate\Foundation\Http\FormRequest;
6 use Illuminate\Validation\Rule;
7
8 class AlatRequest extends FormRequest
9 {
10     public function authorize(): bool
11     {
12         return $this->user()->can('alat.kelola');
13     }
14
15     public function rules(): array
16     {
17         $alatYangDiubah = $this->route('alat');
18
19         return [
20             'kategori_id' => ['required', 'exists:kategori,id'],
21             'kode_alat' => [
22                 'required',
23                 'string',
24                 'max:30',
25                 Rule::unique('alat', 'kode_alat')->ignore($alatYangDiubah),
26             ],
27             'nama' => ['required', 'string', 'max:150'],
28             'deskripsi' => ['nullable', 'string', 'max:1000'],
29             'stok' => ['required', 'integer', 'min:0'],
30             'stok_tersedia' => ['required', 'integer', 'min:0', 'lte:stok'],
31             'kondisi' => ['required', Rule::in(['baik', 'rusak_ringan', 'rusak_berat'])],
32             'foto' => ['nullable', 'image', 'mimes:jpg,jpeg,png', 'max:2048'],
33         ];
34     }
35
36     public function messages(): array
37     {
38         return [
39             'kategori_id.required' => 'Kategori wajib dipilih.',
40             'kode_alat.unique' => 'Kode alat tersebut sudah terdaftar.',
41             'stok_tersedia.lte' => 'Stok tersedia tidak boleh melebihi stok total.',
42             'foto.max' => 'Ukuran foto maksimal 2 MB.',
43             'foto.mimes' => 'Foto harus berformat JPG atau PNG.',
44         ];
45     }
46 }
```

Aturan `'lte:stok'` berarti less than or equal terhadap isian bernama `stok` pada form yang sama. Aturan ini adalah lapis pertama penjaga invarian `0 <= stok_tersedia <= stok`. Lapis keduanya adalah CHECK constraint yang Anda pasang.

Mengapa perlu dua lapis? Validasi di aplikasi memberi pesan yang bisa dibaca pengguna. CHECK constraint menjaga data tetap benar walaupun nanti ada kode lain — stored procedure, trigger, atau query manual — yang menyentuh kolom itu.

3.2.4) Buat controller

Pada terminal buat Controller :

```
php artisan make:controller AlatController --model=Alat
```

Buka file `app\Http\Controllers\AlatController.php`:

```
5 use App\Http\Requests\AlatRequest;
6 use App\Models\Alat;
7 use App\Models\Kategori;
8 use Illuminate\Database\QueryException;
9 use Illuminate\Http\Request;
10 use Illuminate\Support\Facades\Storage;
11
12 class AlatController extends Controller
13 {
14     public function index(Request $request)
15     {
16         $kataKunci = $request->query('cari');
17         $kategoriId = $request->query('kategori_id');
18
19         $daftarAlat = Alat::with('kategori')
20             ->when($kataKunci, function ($query, $kataKunci) {
21                 $query->where(function ($cabang) use ($kataKunci) {
22                     $cabang->where('nama', 'like', '%' . $kataKunci . '%')
23                         ->orWhere('kode_alat', 'like', '%' . $kataKunci . '%');
24                 });
25             })
26             ->when($kategoriId, function ($query, $kategoriId) {
27                 $query->where('kategori_id', $kategoriId);
28             })
29             ->orderBy('kode_alat')
30             ->paginate(10)
31             ->withQueryString();
32
33         $daftarKategori = Kategori::orderBy('nama')->get();
34
35         return view('alat.index', compact(
36             'daftarAlat',
37             'daftarKategori',
38             'kataKunci',
39             'kategoriId'
40         ));
41     }
42
43     public function create()
44     {
45         $alat = new Alat();
46         $daftarKategori = Kategori::orderBy('nama')->get();
47
48         return view('alat.form', compact('alat', 'daftarKategori'));
49     }
50
51     public function store(AlatRequest $request)
52     {
53         $data = $request->validated();
54
55         if ($request->hasFile('foto')) {
56             $namaFile = uniqid() . '.' . $request->file('foto')->extension();
57             $request->file('foto')->storeAs('alat', $namaFile, 'gambar');
58             $data['foto'] = $namaFile;
59         }
60
61         Alat::create($data);
62
63         return redirect()
64             ->route('alat.index')
65             ->with('sukses', 'Data alat berhasil ditambahkan.');
```

```
66     }
67
68     public function edit(Alat $alat)
69     {
70         $daftarKategori = Kategori::orderBy('nama')->get();
71
72         return view('alat.form', compact('alat', 'daftarKategori'));
73     }
74 }
```

```

75     public function update(AlatRequest $request, Alat $alat)
76     {
77         $data = $request->validated();
78
79         if ($request->hasFile('foto')) {
80             $this->hapusFoto($alat->foto);
81             $namaFile = uniqid() . '.' . $request->file('foto')->extension();
82             $request->file('foto')->storeAs('alat', $namaFile, 'gambar');
83             $data['foto'] = $namaFile;
84         }
85
86         $alat->update($data);
87
88         return redirect()
89             ->route('alat.index')
90             ->with('sukses', 'Data alat berhasil diperbarui.');
```

```

91     }
92
93     public function destroy(Alat $alat)
94     {
95         try {
96             $fotoLama = $alat->foto;
97             $alat->delete();
98             $this->hapusFoto($fotoLama);
99         } catch (QueryException $e) {
100             return redirect()
101                 ->route('alat.index')
102                 ->with('gagal', 'Alat tidak dapat dihapus karena sudah pernah dipinjam.');
```

```

103         }
104
105         return redirect()
106             ->route('alat.index')
107             ->with('sukses', 'Data alat berhasil dihapus.');
```

```

108     }
109
110     private function hapusFoto(?string $lokasiFoto): void
111     {
112         if ($lokasiFoto && Storage::disk('gambar')->exists('alat/' . $lokasiFoto)) {
113             Storage::disk('gambar')->delete('alat/' . $lokasiFoto);
114         }
115     }
116 }
```

Empat hal yang membedakan controller ini:

Bagian	Alasan
<code>Alat::with('kategori')</code>	Tanpa <code>with()</code> , menampilkan 10 alat menghasilkan 11 query: satu untuk alat, sepuluh untuk kategorinya masing-masing
Blok <code>if (\$request->hasFile('foto'))</code>	Foto bersifat opsional. Saat mengubah data tanpa memilih foto baru, foto lama harus tetap dipakai
<code>hapusFoto()</code> sebelum menyimpan yang baru	Mencegah berkas menumpuk di server tanpa pernah dipakai
Urutan pada method <code>destroy()</code>	Nama berkas disimpan sebelum baris data dihapus. Setelah baris hilang, <code>\$alat->foto</code> tidak dapat dibaca lagi

Perhatikan juga bahwa `hapusFoto()` diberi tanda private. Method itu alat bantu internal, bukan tujuan sebuah alamat, jadi tidak boleh bisa dipanggil dari route.

Buktikan sendiri manfaat `with()` nanti: buka halaman daftar alat, lalu lihat jumlah query pada bilah Debugbar. Coba hapus `with('kategori')`, muat ulang, dan bandingkan angkanya.

3.2.5) Daftarkan route

Edit file `routes/web.php`:

```
//...
use App\Http\Controllers\KategoriController;

Route::middleware(['auth'])->group(function () {
    //...
    Route::resource('alat', AlatController::class)
        ->except(['show'])
        ->middleware('permission:alat.kelola');
});
```

3.2.6) Buat halaman daftar

Buat `resources/views/alat/index.blade.php`:

```
1  @extends('layouts.utama')
2
3  @section('judul', 'Daftar Alat')
4
5  @section('konten')
6  <div class="d-flex justify-content-between align-items-center mb-3">
7      <h4 class="mb-0">Daftar Alat</h4>
8      <x-tombol-tambah href="route('alat.create')" label="Tambah Alat" />
9  </div>
10
11 <div class="card">
12     <div class="card-body">
13
14         @include('alat.form-pencarian')
15
16         <div class="table-responsive">
17             <table class="table table-striped align-middle">
18                 <thead>
19                     <tr>
20                         <th style="width: 70px">Foto</th>
21                         <th>Kode</th>
22                         <th>Nama</th>
23                         <th>Kategori</th>
24                         <th class="text-center">Stok</th>
25                         <th class="text-center">Tersedia</th>
26                         <th>Kondisi</th>
27                         <th style="width: 160px">Aksi</th>
28                     </tr>
29                 </thead>
30             </table>
31         </div>
32     </div>
33 </div>
```

```

31         <tbody>
32             @foreach ($daftarAlat as $alat)
33                 <tr>
34                     <td>
35                         @if ($alat->foto)
36                             nama }}"
37                                 class="rounded" width="48" height="48" style="object-fit: cover;">
38                         @else
39                             <span class="text-muted small">-</span>
40                         @endif
41                     </td>
42                     <td>{{ $alat->kode_alat }}</td>
43                     <td>{{ $alat->nama }}</td>
44                     <td>{{ $alat->kategori->nama }}</td>
45                     <td class="text-center">{{ $alat->stok }}</td>
46                     <td class="text-center">
47                         <span class="badge bg-{{ $alat->stok_tersedia > 0 ? 'success' : 'secondary' }}">
48                             {{ $alat->stok_tersedia }}
49                         </span>
50                     </td>
51                     <td>{{ str_replace('_', ' ', $alat->kondisi) }}</td>
52                     <td>
53                         <x-tombol-aksi
54                             :ubah="route('alat.edit', $alat)"
55                             :hapus="route('alat.destroy', $alat)"
56                             pesanHapus="Hapus data alat {{ $alat->nama }}" />
57                     </td>
58                 </tr>
59             @empty
60                 <tr>
61                     <td colspan="8" class="text-center text-muted">
62                         Data alat tidak ditemukan.
63                     </td>
64                 </tr>
65             @endforeach
66         </tbody>
67     </table>
68 </div>
69
70     {{ $daftarAlat->links() }}
71
72 </div>
73 </div>
74 @endsection

```

Buat file `resources\views\alat\form-pencarian.blade.php`:

```

1  <form method="GET" action="{{ route('alat.index') }}" class="row g-2 mb-3">
2      <div class="col-md-4">
3          <input type="text" name="cari" class="form-control" placeholder="Cari nama atau kode alat"
4              value="{{ $kataKunci }}">
5      </div>
6      <div class="col-md-3">
7          <select name="kategori_id" class="form-select">
8              <option value="">Semua Kategori</option>
9              @foreach ($daftarKategori as $kategori)
10                 <option value="{{ $kategori->id }}" {{ $kategoriId == $kategori->id ? 'selected' : '' }}>
11                     {{ $kategori->nama }}
12                 </option>
13             @endforeach
14         </select>
15     </div>
16     <div class="col-auto">
17         <button type="submit" class="btn btn-outline-secondary">Saring</button>
18         <a href="{{ route('alat.index') }}" class="btn btn-outline-secondary">Reset</a>
19     </div>
20 </form>

```

3.2.7) Buat halaman form

Buat `resources/views/alat/form.blade.php`:

```
1 @extends('layouts.utama')
2
3 @section('judul', $alat→exists ? 'Ubah Alat' : 'Tambah Alat')
4
5 @section('konten')
6     <div class="row justify-content-center">
7         <div class="col-md-8">
8             <div class="card">
9                 <div class="card-body">
10                     <h5 class="card-title mb-4">
11                         {{ $alat→exists ? 'Ubah Data Alat' : 'Tambah Data Alat' }}
12                     </h5>
13
14                     <form method="POST" action="{{ $alat→exists ? route('alat.update', $alat) : route('alat.store') }}"
15                         enctype="multipart/form-data">
16                         @csrf
17                         @if ($alat→exists)
18                             @method('PUT')
19                         @endif
20
21                         <div class="row">
22                             <div class="col-md-4">
23                                 <x-input name="kode_alat" label="Kode Alat" :value="$alat→kode_alat" />
24                             </div>
25
26                             <div class="col-md-8">
27                                 <x-input name="nama" label="Nama Alat" :value="$alat→nama" />
28                             </div>
29                         </div>
30
31                         <x-select name="kategori_id" label="Kategori" :opsi="$daftarKategori" :value="$alat→kategori_id" keyValue="id"
32                             keyLabel="nama" placeholder="Pilih kategori" />
33
34                         <div class="row">
35                             <div class="col-md-4">
36                                 <x-input name="stok" label="Stok Total" :value="$alat→stok" type="number"
37                                     min="0" />
38                             </div>
39
40                             <div class="col-md-4">
41                                 <x-input name="stok_tersedia" label="Stok Tersedia" :value="$alat→stok_tersedia" type="number"
42                                     min="0" />
43                             </div>
44
45                             <div class="col-md-4">
46                                 <x-select name="kondisi" label="Kondisi" :opsi="[
47                                     ['key' => 'baik', 'label' => 'Baik'],
48                                     ['key' => 'rusak_ringan', 'label' => 'Rusak Ringan'],
49                                     ['key' => 'rusak_berat', 'label' => 'Rusak Berat'],
50                                 ]" :value="$alat→kondisi"
51                                     placeholder="Pilih kondisi" />
52                             </div>
53                         </div>
54
55                         <x-textarea name="deskripsi" label="Deskripsi" rows="3" :value="$alat→deskripsi" />
56
57                         @include('alat.input-foto')
58
59                         <button type="submit" class="btn btn-primary">Simpan</button>
60                         <a href="{{ route('alat.index') }}" class="btn btn-secondary">Batal</a>
61                     </form>
62                 </div>
63             </div>
64         </div>
65     </div>
66 @endsection
```

Atribut `enctype="multipart/form-data"` pada tag form wajib ada. Tanpa itu, berkas foto tidak ikut terkirim, dan `$request->hasFile('foto')` selalu bernilai salah tanpa pesan kesalahan apa pun. Ini kesalahan yang sangat sulit dilacak kalau tidak diketahui sejak awal.

Buat file `resources\views\alat\input-foto.blade.php` :

```
1 <div class="mb-3">
2     <label for="foto" class="form-label">Foto</label>
3     <input type="file" class="form-control @error('foto') is-invalid @enderror" id="foto" name="foto"
4         accept="image/jpeg,image/png">
5     @error('foto')
6         <div class="invalid-feedback">{{ $message }}</div>
7     @enderror
8
9     @if ($alat->foto)
10        <div class="mt-2">
11            
12            <div class="form-text">
13                Biarkan kosong bila tidak ingin mengganti foto.
14            </div>
15        </div>
16    @endif
17 </div>
```

Buat file component `resources\views\components\select.blade.php` :

```
1 @props([
2     'name',
3     'label' => null,
4     'opsi' => [],
5     'value' => null,
6     'keyValue' => 'key',
7     'keyLabel' => 'label',
8     'placeholder' => null,
9 ])
10
11 <div class="mb-3">
12     @if ($label)
13         <label for="{{ $name }}" class="form-label">{{ $label }}</label>
14     @endif
15     <select class="form-select @error($name) is-invalid @enderror" id="{{ $name }}" name="{{ $name }}">
16         @if ($placeholder)
17             <option value="">{{ $placeholder }}</option>
18         @endif
19         @foreach ($opsi as $pilihan)
20             @php
21                 $nilai = $pilihan[$keyValue];
22                 $teks = $pilihan[$keyLabel];
23             @endphp
24             <option value="{{ $nilai }}"
25                 {{ (string) old($name, $value) === (string) $nilai ? 'selected' : '' }}>
26                 {{ $teks }}
27             </option>
28         @endforeach
29     </select>
30     @error($name)
31         <div class="invalid-feedback">{{ $message }}</div>
32     @enderror
33 </div>
```

3.2.8) Aktifkan menu dan uji

Ganti tautan menu Alat di `resources\views\layouts\navbar.blade.php` menjadi `{{ route('alat.index') }}`.

Uji sebagai admin:

No	Percobaan	Hasil yang harus muncul
1	Buka menu Alat	12 alat hasil seeder, terbagi dua halaman
2	Saring berdasarkan kategori Alat Ukur	Tersisa 3 baris
3	Cari dengan kata kunci "AUK"	Tersisa 3 baris, dicocokkan dari kolom kode
4	Tambah alat baru lengkap dengan foto	Tersimpan, foto tampil di daftar
5	Tambah alat dengan kode PKT-001	Ditolak, muncul pesan kode sudah terdaftar
6	Tambah alat dengan stok 5 dan stok tersedia 9	Ditolak, muncul pesan stok tersedia melebihi stok total
7	Ubah alat tanpa memilih foto baru	Foto lama tetap tampil
8	Ubah alat dengan memilih foto baru	Foto berganti, berkas lama hilang dari storage/app/public/alat
9	Hapus alat yang baru dibuat	Berhasil

Halaman Daftar Alat :

The screenshot shows the 'Daftar Alat' page in a web application. The page has a dark header with the title 'Peminjaman Alat' and navigation links for 'Kategori', 'Alat', and 'Pengguna'. The user is logged in as 'Administrator' and can click 'Keluar' (Logout). The main content area is titled 'Daftar Alat' and includes a search bar, a dropdown for 'Semua Kategori', and buttons for 'Saring' (Filter) and 'Reset'. Below the search bar is a table with 12 tools. The table has columns for 'Foto', 'Kode', 'Nama', 'Kategori', 'Stok', 'Tersedia', 'Kondisi', and 'Aksi'. The tools are listed in two pages, with the first page showing 10 results. The user can click 'Ubah' (Edit) or 'Hapus' (Delete) for each tool. The page also shows a pagination bar at the bottom with 'Showing 1 to 10 of 12 results' and a '1 2' button.

Foto	Kode	Nama	Kategori	Stok	Tersedia	Kondisi	Aksi
	AUK-001	Multimeter Digital	Alat Ukur	12	10	baik	Ubah Hapus
-	AUK-002	Jangka Sorong	Alat Ukur	9	9	baik	Ubah Hapus
-	AUK-003	Mistar Baja 30 cm	Alat Ukur	15	15	baik	Ubah Hapus
-	AVI-001	Proyektor Portabel	Perangkat Audio Visual	3	3	baik	Ubah Hapus
-	AVI-002	Tripod Kamera	Perangkat Audio Visual	6	6	baik	Ubah Hapus
-	AVI-003	Kamera Mirrorless	Perangkat Audio Visual	2	2	baik	Ubah Hapus
-	JAR-001	Tang Crimping RJ45	Perangkat Jaringan	10	10	baik	Ubah Hapus
-	JAR-002	LAN Tester	Perangkat Jaringan	5	5	baik	Ubah Hapus
-	JAR-003	Switch 8 Port	Perangkat Jaringan	4	4	baik	Ubah Hapus
-	PKT-001	Obeng Plus Set	Perkakas Tangan	10	10	baik	Ubah Hapus

Showing 1 to 10 of 12 results

1 2

Halaman Form Alat :

Ubah Alat

localhost:8000/alat/4/edit

Guest

Peminjaman Alat

Kategori

Alat

Pengguna

Administrator

Keluar

Ubah Data Alat

Kode Alat

Nama Alat

Kategori

Stok Total

Stok Tersedia

Kondisi

Deskripsi

Foto

Simpan

Batal

AUK-001

Multimeter Digital

Alat Ukur

12

10

Baik

alat ukur elektronik yang dipakai untuk mengukur tegangan listrik, arus listrik, dan hambatan atau resistansi. Alat ini juga sering disebut sebagai AVO meter (Ampere, Volt, Ohm) atau multitester.

Choose File

No file chosen



Biarkan kosong bila tidak ingin mengganti foto.

Request

Timeline

Views 11

Queries 6

Models 7

Gate 6

Cache 1

12.x

28MB

436ms

GET /alat/4/edit

12.x

28MB

436ms

GET /alat/4/edit

12.x

28MB

436ms

GET /alat/4/edit

3.3 CRUD Pengguna dan Penugasan Peran

3.3.1) Buat Form Request

Pada terminal buat Request :

```
php artisan make:request PenggunaRequest
```

Buka file `app\Http\Requests\PenggunaRequest.php` :

```
5 use Illuminate\Foundation\Http\FormRequest;
6 use Illuminate\Validation\Rule;
7
8 class PenggunaRequest extends FormRequest
9 {
10     public function authorize(): bool
11     {
12         return $this->user()->can('user.kelola');
13     }
14
15     public function rules(): array
16     {
17         $penggunaYangDiubah = $this->route('pengguna');
18         $sedangMengubah      = $penggunaYangDiubah !== null;
19
20         return [
21             'nama'          => ['required', 'string', 'max:100'],
22             'username'      => [
23                 'required',
24                 'string',
25                 'max:50',
26                 'alpha_dash',
27                 Rule::unique('users', 'username')->ignore($penggunaYangDiubah),
28             ],
29             'email'         => [
30                 'nullable',
31                 'email',
32                 'max:100',
33                 Rule::unique('users', 'email')->ignore($penggunaYangDiubah),
34             ],
35             'no_telp'       => ['nullable', 'string', 'max:20'],
36             'password'      => [
37                 $sedangMengubah ? 'nullable' : 'required',
38                 'string',
39                 'min:8',
40                 'confirmed',
41             ],
42             'peran'         => ['required', 'exists:roles,name'],
43             'is_aktif'      => ['required', 'boolean'],
44         ];
45     }
46
47     public function messages(): array
48     {
49         return [
50             'username.unique'      => 'Nama pengguna tersebut sudah dipakai.',
51             'username.alpha_dash' => 'Nama pengguna hanya boleh berisi huruf, angka, garis bawah, dan tanda hubung.',
52             'password.confirmed'  => 'Konfirmasi kata sandi tidak cocok.',
53             'password.min'        => 'Kata sandi minimal 8 karakter.',
54             'peran.required'      => 'Peran wajib dipilih.',
55         ];
56     }
57 }
```

Baris paling menarik di berkas ini adalah aturan `password`. Nilainya berubah tergantung keadaan: `required` saat menambah pengguna baru, `nullable` saat mengubah. Dengan begitu, Admin yang hanya ingin memperbaiki nomor telepon tidak dipaksa mengetik ulang kata sandi.

Aturan `confirmed` menuntut adanya isian bernama `password_confirmation` pada form. Nama itu wajib persis, tidak boleh diterjemahkan, karena Laravel mencarinya berdasarkan pola `namafield_confirmation`.

3.3.2) Buat controller

Pada terminal buat Controller :

```
php artisan make:controller PenggunaController --model=User
```

Buka file `app\Http\Controllers\PenggunaController.php` :

```
5 use App\Http\Requests\PenggunaRequest;
6 use App\Models\User;
7 use Illuminate\Http\Request;
8 use Spatie\Permission\Models\Role;
9
10 class PenggunaController extends Controller
11 {
12     public function index(Request $request)
13     {
14         $kataKunci = $request->query('cari');
15         $peran = $request->query('peran');
16
17         $daftarPengguna = User::with('roles')
18             ->when($kataKunci, function ($query, $kataKunci) {
19                 $query->where(function ($cabang) use ($kataKunci) {
20                     $cabang->where('nama', 'like', '%' . $kataKunci . '%')
21                         ->orWhere('username', 'like', '%' . $kataKunci . '%');
22                 });
23             })
24             ->when($peran, function ($query, $peran) {
25                 $query->role($peran);
26             })
27             ->orderBy('nama')
28             ->paginate(10)
29             ->withQueryString();
30
31         $daftarPeran = Role::orderBy('name')->get();
32
33         return view('pengguna.index', compact(
34             'daftarPengguna',
35             'daftarPeran',
36             'kataKunci',
37             'peran'
38         ));
39     }
40
41     public function create()
42     {
43         $pengguna = new User();
44         $daftarPeran = Role::orderBy('name')->get();
45
46         return view('pengguna.form', compact('pengguna', 'daftarPeran'));
47     }
48 }
```

```

49     public function store(PenggunaRequest $request)
50     {
51
52         $data = $request->validated();
53
54         $pengguna = User::create([
55             'nama'      => $data['nama'],
56             'username' => $data['username'],
57             'email'     => $data['email'] ?? null,
58             'no_telp'   => $data['no_telp'] ?? null,
59             'password' => $data['password'],
60             'is_aktif' => $data['is_aktif'],
61         ]);
62
63         $pengguna->syncRoles([$data['peran']]);
64
65         return redirect()
66             ->route('pengguna.index')
67             ->with('sukses', 'Pengguna berhasil ditambahkan.');
```

```

70     public function edit(User $pengguna)
71     {
72         $daftarPeran = Role::orderBy('name')->get();
73
74         return view('pengguna.form', compact('pengguna', 'daftarPeran'));
75     }
76
77     public function update(PenggunaRequest $request, User $pengguna)
78     {
79         $data = $request->validated();
80
81         if ($this->diriSendiri($pengguna) && !$data['is_aktif']) {
82             return back()
83                 ->with('gagal', 'Anda tidak dapat menonaktifkan akun Anda sendiri.')
84                 ->withInput();
85         }
86
87         $pengguna->nama      = $data['nama'];
88         $pengguna->username  = $data['username'];
89         $pengguna->email     = $data['email'] ?? null;
90         $pengguna->no_telp   = $data['no_telp'] ?? null;
91         $pengguna->is_aktif  = $data['is_aktif'];
92
93         if (! empty($data['password'])) {
94             $pengguna->password = $data['password'];
95         }
96
97         $pengguna->save();
98         $pengguna->syncRoles([$data['peran']]);
99
100        return redirect()
101            ->route('pengguna.index')
102            ->with('sukses', 'Data pengguna berhasil diperbarui.');
```

```

103    }
104
105    public function destroy(User $pengguna)
106    {
107        if ($this->diriSendiri($pengguna)) {
108            return redirect()
109                ->route('pengguna.index')
110                ->with('gagal', 'Anda tidak dapat menghapus akun Anda sendiri.');
```

Tentang larangan diri sendiri. Pengguna tidak boleh menghapus atau menonaktifkan akunnya sendiri. Bayangkan bila aturan ini tidak ada: satu-satunya Admin menonaktifkan akunnya, lalu tidak ada seorang pun yang bisa mengaktifkannya kembali karena hanya Admin yang punya izin itu. Aplikasi terkunci selamanya. Kasus seperti ini disebut **lockout**, dan mencegahnya jauh lebih mudah daripada memperbaikinya.

Tentang password saat mengubah. Perhatikan blok `if (! empty($data['password']))`. Kalau isian dibiarkan kosong, kolom password tidak disentuh sama sekali. Ingat juga bahwa `Hash::make()` tidak dipanggil di mana pun — cast `'password' => 'hashed'` pada model User sudah menanganinya.

Tentang `syncRoles()`. Method ini mengganti seluruh peran pengguna dengan daftar baru. Berbeda dengan `assignRole()` yang hanya menambahkan, sehingga bila dipakai saat mengubah data, pengguna bisa berakhir memiliki dua peran sekaligus.

Pemeriksaan ini sebenarnya lapis kedua, karena foreign key `peminjaman.user_id` sudah memakai `restrictOnDelete()`. Bedanya, pemeriksaan di controller menghasilkan pesan yang menjelaskan **apa yang harus dilakukan sebagai gantinya** — menonaktifkan akun. Pesan yang memberi jalan keluar selalu lebih baik daripada pesan yang hanya melarang.

3.3.3) Tambahkan relasi pada model

Method `destroy()` di atas memanggil `peminjamanDiajukan()`. Tambahkan relasinya di `app/Models/User.php`:

```
//...
class User extends Authenticatable
{
    //...
    public function peminjamanDiajukan()
    {
        return $this->hasMany(Peminjaman::class, 'user_id');
    }
}
```

3.3.4) Daftarkan route

Edit file `routes/web.php`:

```
//...
use App\Http\Controllers\PenggunaController;

Route::middleware(['auth'])->group(function () {
    //...
    Route::resource('pengguna', PenggunaController::class)
        ->except(['show'])
        ->middleware('permission:user.kelola');
});
```

Nama parameter `{pengguna}` sengaja berbeda dari nama model `User`. Agar Laravel tetap dapat mencari datanya, nama variabel di controller harus sama: `User $pengguna`. Kesamaan nama parameter route dengan nama variabel itulah yang menjadi patokannya, bukan nama modelnya.

3.3.5) Buat halaman daftar

Buat `resources/views/pengguna/daftar.blade.php`:

```
1 @extends('layouts.utama')
2
3 @section('judul', 'Daftar Pengguna')
4
5 @section('konten')
6 <div class="d-flex justify-content-between align-items-center mb-3">
7     <h4 class="mb-0">Daftar Pengguna</h4>
8     <x-tombol-tambah href="{{ route('pengguna.create') }}" label="Tambah Pengguna" />
9 </div>
10
11 <div class="card">
12     <div class="card-body">
13
14         @include('pengguna.form-pencarian')
15
16         @include('pengguna.table')
17
18         {{ $daftarPengguna->links() }}
19
20     </div>
21 </div>
22 @endsection
```

Buat file `resources\views\pengguna\form-pencarian.blade.php` :

```
1 <form method="GET" action="{{ route('pengguna.index') }}" class="row g-2 mb-3">
2     <div class="col-md-4">
3         <input type="text" name="cari" class="form-control" placeholder="Cari nama atau nama pengguna"
4             value="{{ $kataKunci }}">
5     </div>
6     <div class="col-md-3">
7         <select name="peran" class="form-select">
8             <option value="">Semua Peran</option>
9             @foreach ($daftarPeran as $pilihanPeran)
10                 <option value="{{ $pilihanPeran->name }}" {{ $peran == $pilihanPeran->name ? 'selected' : '' }}>
11                     {{ ucfirst($pilihanPeran->name) }}
12                 </option>
13             @endforeach
14         </select>
15     </div>
16     <div class="col-auto">
17         <button type="submit" class="btn btn-outline-secondary">Filter</button>
18         <a href="{{ route('pengguna.index') }}" class="btn btn-outline-secondary">Reset</a>
19     </div>
20 </form>
```

Buat file `resources\views\pengguna\table.blade.php` :

```
1 <div class="table-responsive">
2     <table class="table table-striped align-middle">
3         <thead>
4             <tr>
5                 <th style="width: 60px">No</th>
6                 <th>Nama</th>
7                 <th>Nama Pengguna</th>
8                 <th>Peran</th>
9                 <th>Telepon</th>
10                <th>Status</th>
11                <th style="width: 160px">Aksi</th>
12            </tr>
13        </thead>
```

```

15     <tbody>
16         @forelse ($daftarPengguna as $nomor => $pengguna)
17             <tr>
18                 <td>{{ $daftarPengguna->firstItem() + $nomor }}</td>
19                 <td>{{ $pengguna->nama }}</td>
20                 <td>{{ $pengguna->username }}</td>
21                 <td>
22                     @foreach ($pengguna->roles as $peranPengguna)
23                         <span class="badge bg-info text-dark">
24                             {{ ucfirst($peranPengguna->name) }}
25                         </span>
26                     @endforeach
27                 </td>
28                 <td>{{ $pengguna->no_telp ?: '-' }}</td>
29                 <td>
30                     <span class="badge bg-{{ $pengguna->is_aktif ? 'success' : 'secondary' }}">
31                         {{ $pengguna->is_aktif ? 'Aktif' : 'Nonaktif' }}
32                     </span>
33                 </td>
34                 <td>
35                     @php
36                         $hapus = $pengguna->id == auth()->id() ? route('pengguna.destroy', $pengguna) : null;
37                     @endphp
38
39                     <x-tombol-aksi :ubah="route('pengguna.edit', $pengguna)" :hapus="$hapus"
40                         pesanHapus="Yakin ingin menghapus pengguna {{ $pengguna->nama }}" />
41
42                 </td>
43             </tr>
44         @empty
45             <tr>
46                 <td colspan="7" class="text-center text-muted">
47                     Data pengguna tidak ditemukan.
48                 </td>
49             </tr>
50         @endforelse
51     </tbody>
52 </table>
53 </div>

```

Tombol Hapus disembunyikan untuk akun sendiri. Ini hanya kenyamanan tampilan; penjagaan sebenarnya tetap ada di controller, karena tombol yang disembunyikan masih bisa ditembus dengan mengirim permintaan langsung.

3.3.6) Buat halaman form

Buat `resources/views/pengguna/form.blade.php`:

```

1  @extends('layouts.utama')
2
3  @section('judul', $pengguna->exists ? 'Ubah Pengguna' : 'Tambah Pengguna')
4
5  @section('konten')
6      <div class="row justify-content-center">
7          <div class="col-md-8">
8              <div class="card">
9                  <div class="card-body">
10                     <h5 class="card-title mb-4">
11                         {{ $pengguna->exists ? 'Ubah Data Pengguna' : 'Tambah Data Pengguna' }}
12                     </h5>
13
14                     <form method="POST"
15                         action="{{ $pengguna->exists ? route('pengguna.update', $pengguna) : route('pengguna.store') }}">
16                         @csrf
17                         @if ($pengguna->exists)
18                             @method('PUT')
19                         @endif
20

```

```

20
21     <div class="row">
22         <div class="col-md-6">
23             <x-input label="Nama Lengkap" name="nama" :value="$pengguna->nama" required />
24         </div>
25
26         <div class="col-md-6">
27             <x-input label="Nama Pengguna" name="username" :value="$pengguna->username" required />
28         </div>
29     </div>
30
31     <div class="row">
32         <div class="col-md-6">
33             <x-input label="Email" name="email" :value="$pengguna->email" type="email" required />
34         </div>
35
36         <div class="col-md-6">
37             <x-input label="Nomor Telepon" name="no_telp" :value="$pengguna->no_telp" />
38         </div>
39     </div>
40
41     @if ($pengguna->exists)
42         <span class="text-muted small">(Password kosongkan bila tidak diganti)</span>
43     @endif
44
45     <div class="row">
46         <div class="col-md-6">
47             <x-input label="Kata Sandi" name="password" type="password" />
48         </div>
49
50         <div class="col-md-6">
51             <x-input label="Konfirmasi Kata Sandi" name="password_confirmation" type="password" />
52         </div>
53     </div>
54
55     <div class="row">
56         <div class="col-md-6 mb-3">
57             <label for="peran" class="form-label">Peran</label>
58             <select class="form-select @error('peran') is-invalid @enderror" id="peran"
59                 name="peran" required>
60                 <option value="">-- Pilih Peran --</option>
61                 @foreach ($daftarPeran as $pilihanPeran)
62                     <option value="{{ $pilihanPeran->name }}"
63                         {{ old('peran', $pengguna->roles->first()->name) == $pilihanPeran->name ? 'selected' : '' }}>
64                         {{ ucfirst($pilihanPeran->name) }}
65                     </option>
66                 @endforeach
67             </select>
68             @error('peran')
69                 <div class="invalid-feedback">{{ $message }}</div>
70             @enderror
71         </div>
72
73         <div class="col-md-6 mb-3">
74             <x-select label="Aktif" name="is_aktif" :value="$pengguna->is_aktif" placeholder="Pilih status aktif"
75                 :opsi="[['key' => 1, 'label' => 'Aktif'], ['key' => 0, 'label' => 'Nonaktif']]"/>
76         </div>
77     </div>
78
79     <button type="submit" class="btn btn-primary">Simpan</button>
80     <a href="{{ route('pengguna.index') }}" class="btn btn-secondary">Batal</a>
81 </form>
82 </div>
83 </div>
84 @endsection
85

```

Baris `$pengguna->roles->first()->name` memakai tanda tanya sebelum panah. Tanda itu berarti: kalau tidak ada peran sama sekali, hasilnya kosong, bukan halaman error. Diperlukan karena pada form tambah, objek pengguna masih baru dan belum punya peran apa pun.

3.3.7) Aktifkan menu dan uji

Ganti tautan menu Pengguna di navbar menjadi `{{ route('pengguna.index') }}`.

Uji sebagai admin:

No	Percobaan	Hasil yang harus muncul
1	Buka menu Pengguna	3 akun hasil seeder dengan lensana peran masing-masing
2	Saring berdasarkan peran petugas	Tersisa 1 baris
3	Tambah peminjam baru	Tersimpan, peran tampil sebagai lensana
4	Periksa kolom password di phpMyAdmin	Berupa hash diawali \$2y\$, bukan teks polos
5	Tambah pengguna dengan username admin	Ditolak, muncul pesan sudah dipakai
6	Tambah pengguna dengan konfirmasi kata sandi berbeda	Ditolak, muncul pesan konfirmasi tidak cocok
7	Ubah nomor telepon tanpa mengisi kata sandi	Berhasil, dan akun masih bisa dipakai masuk dengan kata sandi lama
8	Ubah peran peminjam baru menjadi petugas	Berhasil, lensana berubah, dan setelah masuk menunya ikut berubah
9	Setel akun sendiri menjadi Nonaktif	Ditolak, muncul pesan merah
10	Nonaktifkan akun peminjam lalu coba masuk dengan akun itu	Muncul pesan "Akun Anda dinonaktifkan"

Halaman Daftar Pengguna :

The screenshot displays the 'Daftar Pengguna' (User List) page. At the top, there's a navigation bar with 'Peminjaman Alat', 'Kategori', 'Alat', and 'Pengguna'. The 'Pengguna' section is active. Below the navigation bar, there's a header with 'Daftar Pengguna' and a 'Tambah Pengguna' button. The main content area features a search bar with the placeholder 'Cari nama atau nama pengguna', a dropdown for 'Semua Peran', and 'Filter' and 'Reset' buttons. Below this is a table with the following data:

No	Nama	Nama Pengguna	Peran	Telepon	Status	Aksi
1	Administrator	admin	Admin	081200000001	Aktif	Ubah
2	Petugas Laboratorium	petugas	Petugas	081200000002	Aktif	Ubah Hapus
3	Siswa Peminjam	peminjam	Peminjam	081200000003	Aktif	Ubah Hapus

At the bottom of the page, there's a status bar showing various metrics: Request, Timeline, Views (10), Queries (10), Models (11), Gate (6), Cache (1), and other system information like '12.x', '28MB', '529ms', and 'GET /pengguna'.

Halaman Form Pengguna :

Ubah Pengguna

localhost:8000/pengguna/2/edit

Guest

Peminjaman Alat

Kategori

Alat

Pengguna

Administrator

Keluar

Ubah Data Pengguna

Nama Lengkap

Petugas Laboratorium

Nama Pengguna

petugas

Email

petugas@sekolah.sch.id

Nomor Telepon

081200000002

(Password kosongkan bila tidak diganti)

Kata Sandi

Konfirmasi Kata Sandi

Peran

Petugas

Aktif

Aktif

Simpan

Batal

Request

Timeline

Views 10

Queries 9

Models 7

Gate 6

Cache 1

12.x

28MB

444ms

GET /pengguna/2/edit